

OCI Terraform & Kubernetes Deployment

Lab 3 — Containerized Application on Oracle Kubernetes Engine (OKE)

1. Project Overview

This document describes the end-to-end deployment of a containerized static web application onto Oracle Cloud Infrastructure (OCI) using Terraform-provisioned networking and an Oracle Kubernetes Engine (OKE) cluster. The infrastructure is built from two reusable Terraform modules (subnet and oke), and the application is packaged as a Docker image, pushed to the Oracle Cloud Infrastructure Registry (OCIR), and deployed to Kubernetes with persistent storage and public load-balanced ingress.

2. Target OCI Architecture

The target architecture is a production-grade, highly available OKE setup built from reusable Terraform modules, deployed in region me-jeddah-1 within a 10.0.0.0/16 VCN.

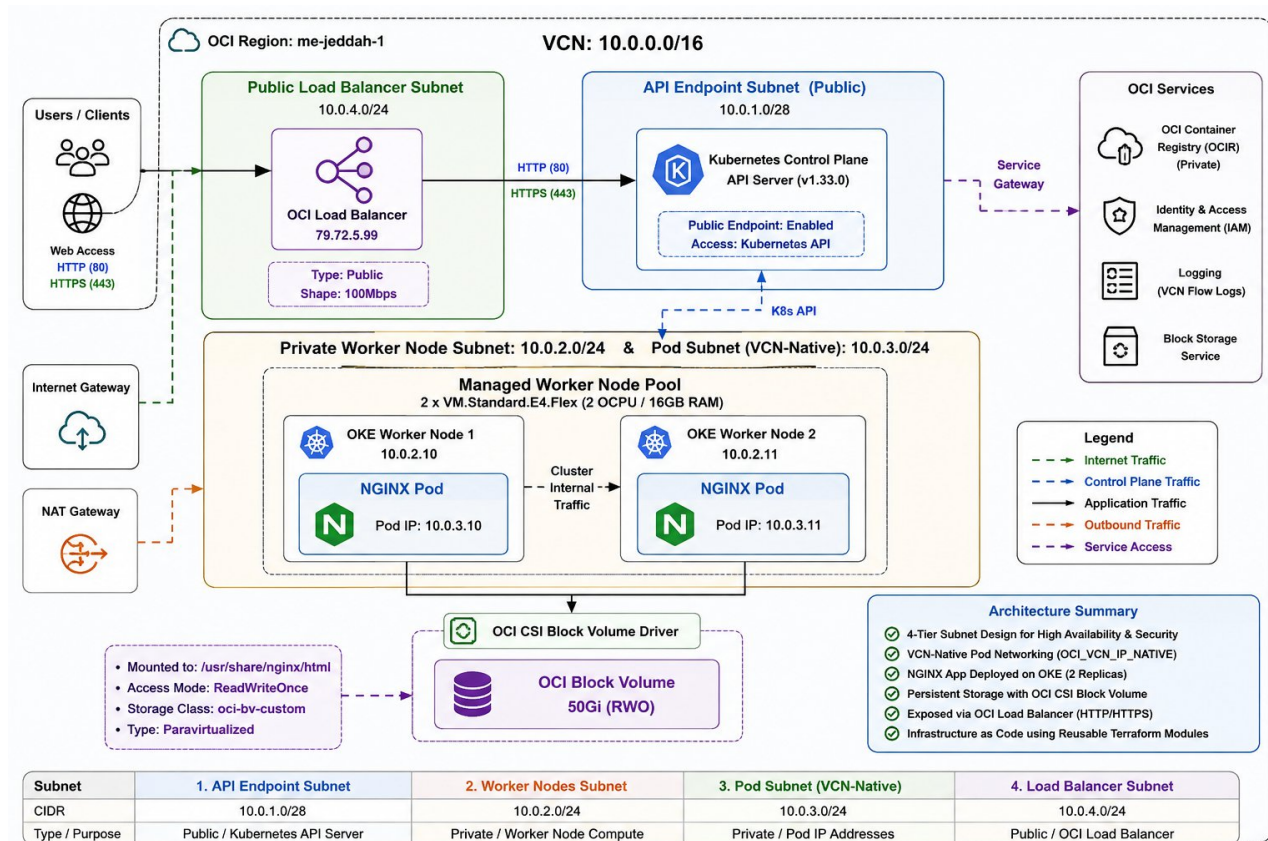


Figure 1 — End-to-end target architecture: public ingress through the OCI Load Balancer and API Endpoint subnet, a private worker/pod tier running the NGINX deployment on OKE, CSI-backed block storage, and OCI service access via the Service Gateway.

2.1 Key Architectural Highlights

- 4-tier subnet design for high availability & security: api_endpoint, worker_nodes, pod_subnet, load_balancer — each isolated with strict Security Lists.
- VCN-Native Pod Networking: pods receive native IP addresses directly from the OCI VCN CIDR (OCI_VCN_IP_NATIVE).
- NGINX application deployed on OKE (2 replicas in design; see Section 6.2 for the RWO constraint that reduced this to 1 in the running deployment).

- Persistent storage via the OCI CSI Block Volume driver — a PersistentVolumeClaim (oci-block-volume-pvc), 50Gi, ReadWriteOnce, storage class oci-bv-custom, mounted at /usr/share/nginx/html.
- Exposed via the OCI Load Balancer over HTTP (80) and HTTPS (443).
- Infrastructure as Code using reusable Terraform modules (subnet, oke).
- OCI Registry Integration (OCIR): a private image repository hosts the containerized application image.

2.2 Subnet Reference

Subnet	CIDR	Type / Purpose
1. API Endpoint Subnet	10.0.0.0/28	Public / Kubernetes API Server
2. Worker Nodes Subnet	10.0.10.0/24	Private / Worker Node Compute
3. Pod Subnet (VCN-Native)	10.0.32.0/19	Private / Pod IP Addresses
4. Load Balancer Subnet	10.0.20.0/24	Public / OCI Load Balancer

Note: the diagram's illustrative CIDR labels (10.0.1.0/28, 10.0.2.0/24, 10.0.3.0/24, 10.0.4.0/24) correspond to the actual provisioned values above, confirmed against the OCI Console in Section 6.2.

3. Application Dependency Assessment

The source application is a lightweight, high-performance static web front end serving as the Cloud Build Track dashboard — an entry point for monitoring container health and cloud infrastructure deployment status.

3.1 Runtime & Dependency Matrix

Layer	Component	Requirement / Specification	Purpose
Web Server	NGINX	>= 1.25.0 (Alpine Linux build)	Serving HTTP requests, static asset hosting, reverse proxying
Runtime Container	Docker Engine	>= 24.0.0 / OCI Runtime	Container isolation and image packaging
Port Bindings	TCP 80 / TCP 443	Exposed internal port 80	Receiving web traffic from Kubernetes Service
Health Check	wget HTTP probe	GET / returning HTTP 200 OK	Container liveness and readiness probing
Persistent Storage	OCI CSI Volume	50Gi paravirtualized block volume	Mounted at /usr/share/nginx/html

3.2 Network & Security Requirements

- Inbound access: HTTP (TCP 80) permitted from the OCI Load Balancer subnet only.
- Outbound access: unrestricted egress for package updates and OCI API communication.
- Security profile: capable of non-root container process execution; compatible with a read-only root filesystem.

4. Terraform Modules & File Explanation

4.1 Subnet Module (modules/subnet)

A reusable, dynamic module encapsulating a VCN Subnet, Route Table, Security List, and optional VCN Flow Logs, eliminating duplicated networking code across the four subnets.

Key Resources (main.tf)

- `oci_core_route_table.this` — a dynamic "route_rules" block constructs arbitrary routing rules (Internet Gateway or NAT Gateway routes) from input variables.
- `oci_core_security_list.this` — dynamic "ingress_security_rules" and "egress_security_rules" blocks construct firewall rules for TCP, UDP, ICMP, or custom protocols.
- `oci_core_subnet.this` — the core VCN subnet resource linked to the created route table and security list; supports public/private flags (`prohibit_public_ip_on_vnic`) and custom DNS labels.
- `oci_logging_log.subnet_flow_log` — conditionally created via `count = var.enable_subnet_logs && var.log_group_id != "" ? 1 : 0`.

Inputs / Outputs

Category	Names
Inputs	<code>compartment_id</code> , <code>vcn_id</code> , <code>subnet_name</code> , <code>cidr_block</code> , <code>dns_label</code> , <code>prohibit_public_ip_on_vnic</code> , <code>enable_subnet_logs</code> , <code>log_group_id</code> , <code>route_rules</code> , <code>ingress_rules</code> , <code>egress_rules</code>
Outputs	<code>subnet_id</code> , <code>subnet_cidr</code> , <code>route_table_id</code> , <code>security_list_id</code> , <code>log_id</code>
Versions	Terraform $\sim$ > 1.3, oracle/oci provider $\geq$ 5.0

4.2 OKE Module (modules/oke)

Provisions the Oracle Container Engine for Kubernetes (OKE) cluster with VCN-native pod networking and a managed worker node pool matching specific compute and OS requirements.

Key Resources (main.tf)

- `oci_containerengine_cluster.this` — creates the OKE cluster configured with VCN-Native Pod Networking (OCI_VCN_IP_NATIVE) so pods acquire native OCI VCN IP addresses.
- `oci_containerengine_node_pool.this` — a managed worker node pool with flexible compute shapes (OCPUs and memory), placement across availability domains, and pod network attachments (`node_pool_pod_network_option_details`).

Inputs / Outputs

Category	Names
Inputs	<code>cluster_name</code> , <code>kubernetes_version</code> , <code>api_endpoint_subnet_id</code> , <code>is_api_endpoint_public</code> , <code>lb_subnet_ids</code> , <code>node_subnet_id</code> , <code>pod_subnet_id</code> , <code>node_shape</code> , <code>node_ocpus</code> , <code>node_memory_in_gbs</code> , <code>node_count</code> , <code>node_image_id</code> , <code>ssh_public_key</code>
Outputs	<code>cluster_id</code> , <code>cluster_name</code> , <code>kubernetes_version</code> , <code>node_pool_id</code> , <code>cluster_endpoints</code> , <code>kubeconfig_command</code>

4.3 Kubernetes Manifests (k8s/)

The manifests fulfill the lab requirement of deploying an application, attaching an external OCI block volume, and exposing it through an OCI Load Balancer.

File	Purpose	Details
storageclass.yaml	OCI Block Volume StorageClass	Uses the OCI CSI Driver (blockvolume.csi.oraclecloud.com); volumeBindingMode: WaitForFirstConsumer; paravirtualized attachment.
pvc.yaml	PersistentVolumeClaim	Requests a 50Gi block volume from oci-bv-custom; accessModes: [ReadWriteOnce].
deployment.yaml	NGINX application deployment	Mounts PVC oci-block-volume-pvc at /usr/share/nginx/html; replicas set to 1 (see Section 6.2).
service.yaml	External exposure	type: LoadBalancer with OCI annotation service.beta.kubernetes.io/oci-load-balancer-shape: "100Mbps"; ports 80 and 443 both map to targetPort 80.

4.4 Workspace File Structure

```
Labs/Lab-3/
├── modules/
│   ├── subnet/ (main.tf, variables.tf, outputs.tf, versions.tf)
│   └── oke/ (main.tf, variables.tf, outputs.tf, versions.tf)
├── k8s/
│   ├── storageclass.yaml
│   ├── pvc.yaml
│   ├── deployment.yaml
│   └── service.yaml
├── provider.tf      - OCI provider config + Availability Domains
├── variables.tf     - global root variables
├── vcn.tf           - VCN, IGW, NAT GW, Service GW, log group
├── subnets.tf      - instantiates module "subnet" 4 times
├── oke.tf           - instantiates module "oke"
├── outputs.tf       - VCN ID, subnet IDs, cluster ID, kubeconfig cmd
├── terraform.tfvars - active variable values
└── README.md
```

vcn.tf provisions the core non-repeated network infrastructure: the main VCN (10.0.0.0/16), an Internet Gateway for public subnets, a NAT Gateway for private subnets, a Service Gateway for private OCI service access, and a log group for VCN Flow Logs.

subnets.tf instantiates the subnet module four times — api_endpoint_subnet, worker_nodes_subnet, pod_subnet, and load_balancer_subnet — while oke.tf wires the OKE module to the outputs of vcn.tf and subnets.tf.

5. Setup & Execution Guide — Complete Commands

5.1 Infrastructure Provisioning

```
cd /home/ali_hamad/terraform/Labs/Lab-3
terraform init
terraform validate
terraform plan
terraform apply -auto-approve
```

5.2 OKE Authentication & Connecting

```
oci ce cluster create-kubeconfig --cluster-id <cluster_id> \
  --file ~/.kube/config --region me-jeddah-1 --token-version 2.0.0
kubectl get nodes -o wide
```

5.3 Docker Containerization & Registry Push

```
docker build -t ejada-app:local .
docker tag ejada-app:local alihamad2411/ejada-cloud-app:v1.0.0
docker login -u alihamad2411
docker push alihamad2411/ejada-cloud-app:v1.0.0
docker images | grep ejada
```

5.4 Kubernetes Application Deployment

```
kubectl apply -f k8s/storageclass.yaml
kubectl apply -f k8s/pvc.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

5.5 Rollout Commands

```
kubectl rollout restart deployment web-app-deployment
kubectl rollout status deployment web-app-deployment --timeout=60s
kubectl delete deployment web-app-deployment --now
kubectl apply -f k8s/deployment.yaml
```

5.6 Connectivity Testing Commands

```
kubectl get pods,svc,pvc,sc -o wide
curl -I http://79.72.5.99
curl -v --connect-timeout 5 http://79.72.5.99
curl -v -k https://79.72.5.99
```

6. Output Verification

The following evidence confirms that the infrastructure was provisioned as designed and that the application is healthy and reachable.

6.1 Live Application Dashboard

The deployed application exposes a status dashboard confirming container specification, OCI deployment configuration, and the completion state of each deliverable.

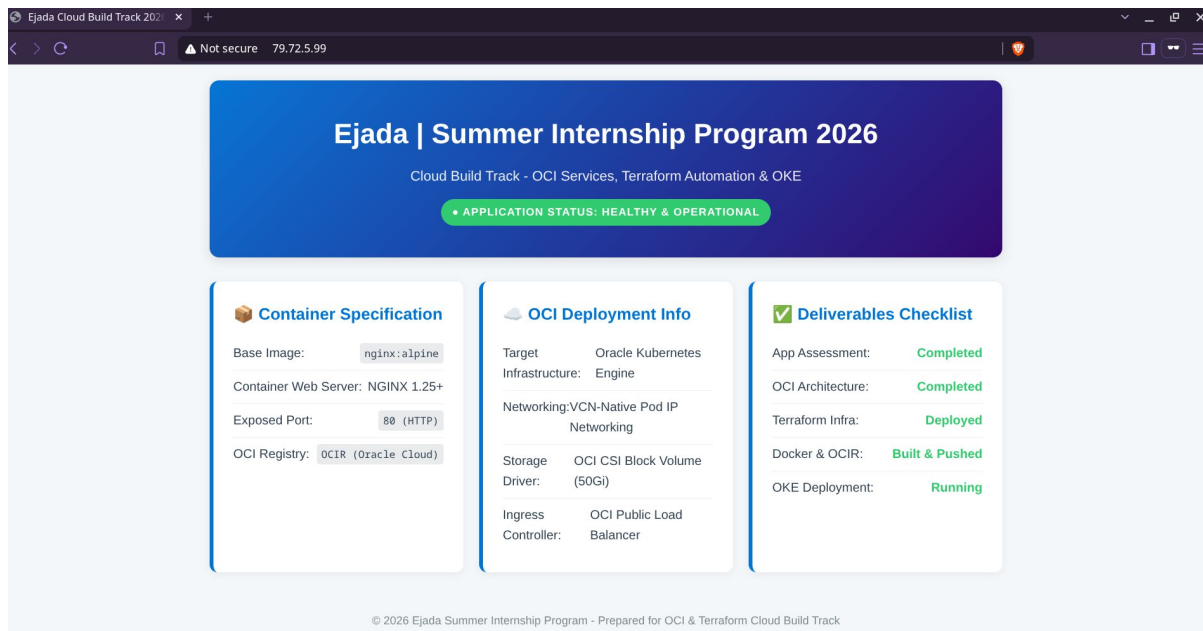


Figure 2 — Ejada Cloud Build Track dashboard served from the OKE-hosted NGINX container, reachable at the public Load Balancer IP (79.72.5.99).

- Base image: nginx:alpine, running NGINX 1.25+, exposing port 80 (HTTP).
- Application image is stored in OCIR (Oracle Cloud Infrastructure Registry).
- Persistent storage: OCI CSI Block Volume driver, 50Gi.
- Ingress: OCI Public Load Balancer.
- All five deliverables — App Assessment, OCI Architecture, Terraform Infra, Docker & OCIR, OKE Deployment — are complete and the workload is Running.

6.2 Provisioned Networking — Console Verification

The screenshots below confirm the subnets and route tables as actually provisioned in the OCI Console for compartment intern-15-ali-hamad-cmp.

Subnets

Applied filters: Compartment Intern-15-all-hamad-cmp

Name	State	IPv4 CIDR Block	Oracle-allocated IPv6 Prefix	BYOIP IPv6 Prefixes	ULA IPv6 Prefixes	Subnet Access	Created	
lab3-worker-nodes-subnet	Available	10.0.10.0/24	—			Private (Regional)	Aug 19, 2026, 13:30:15 UTC	...
lab3-pod-subnet	Available	10.0.32.0/19	—			Private (Regional)	Aug 19, 2026, 13:30:14 UTC	...
lab3-lb-subnet	Available	10.0.20.0/24	—			Public (Regional)	Aug 19, 2026, 13:30:12 UTC	...
lab3-api-endpoint-subnet	Available	10.0.0.0/28	—			Public (Regional)	Aug 19, 2026, 13:30:12 UTC	...

Figure 3 — Four subnets provisioned by the subnet module: lab3-worker-nodes-subnet (10.0.10.0/24, Private), lab3-pod-subnet (10.0.32.0/19, Private), lab3-lb-subnet (10.0.20.0/24, Public), lab3-api-endpoint-subnet (10.0.0.0/28, Public).

Route Table

Add Route Rules Actions

☐	Destination	Target Type	Target	Route Type	Description	
☐	0.0.0.0/0	NAT Gateway	lab3-nat-gw	Static	Outbound route for pods via NAT Gateway	...
☐	All JED Services In Oracle Services Network	Service Gateway	lab3-service-gw	Static	Route pods to OCI Services Network	...

Page 1 of 1 (1 - 2 of 2 total items)

Figure 4 — Route table associated with the load balancer / API-facing subnet, routing 0.0.0.0/0 via the NAT Gateway (lab3-nat-gw) and Oracle Services traffic via the Service Gateway (lab3-service-gw).

lab3-worker-nodes-subnet-route-table Available

Add Route Rules Actions

☐	Destination	Target Type	Target	Route Type	Description	
☐	0.0.0.0/0	NAT Gateway	lab3-nat-gw	Static	Route to NAT Gateway for outbound internet access	...
☐	All JED Services In Oracle Services Network	Service Gateway	lab3-service-gw	Static	Route to Oracle Services Network	...

Figure 5 — lab3-worker-nodes-subnet-route-table: outbound internet access for worker nodes via the NAT Gateway, and access to the Oracle Services Network via the Service Gateway.

7. Problems Faced & Troubleshooting

7.1 NGINX 403 Forbidden Error on Application Load

Problem: Accessing the Load Balancer IP returned a 403 Forbidden error. Mounting the empty OCI Block Volume at `/usr/share/nginx/html` overwrote the container's built-in `index.html`.

Fix: Added an `initContainers` block in `k8s/deployment.yaml` that runs `cp -f /usr/share/nginx/html/index.html /mnt/pvc/index.html` to seed the newly mounted volume before NGINX starts.

7.2 Pods Stuck in Init State (Volume Attachment Conflict)

Problem: With `replicas: 2`, the OCI Block Volume's `ReadWriteOnce` access mode allowed it to bind to only one worker node at a time, so the second pod stalled waiting for volume attachment.

Fix: Changed the deployment specification to `replicas: 1` in `k8s/deployment.yaml` to respect RWO attachment limits.

7.3 Load Balancer Timeout (ERR_TIMED_OUT) in Browsers

Problem: Modern browsers automatically upgrade explicit IPs to `https://`. Since the Load Balancer only listened on HTTP port 80, the HTTPS request timed out; additionally, `worker_nodes_subnet` lacked ingress permissions from `load_balancer_subnet`.

Fix: Added an ingress rule to `worker_nodes_subnet` allowing traffic from `var.lb_subnet_cidr`, and exposed both port 80 and port 443 in `k8s/service.yaml`, mapping both to `targetPort: 80` to safely catch automatic browser HTTPS requests.

7.4 Node Pool Shape and Image Compatibility Errors

Problem: The node pool failed to provision because standard Oracle-Linux-8.10 images were incompatible with OKE managed nodes on `VM.Standard.E4.Flex` shapes.

Fix: Integrated a dynamic `oci_containerengine_node_pool_option` data source in `modules/oke/main.tf` to automatically look up and select a valid Oracle-Linux-8...-OKE image compatible with the chosen shape and Kubernetes version.